# Statistics for Data Analysis with Python

## Exercise Guide

# CONTENTS

Module 1 – Central tendency, variation, and outliers ..... 3
Exercise 1 – Python ..... 3
Exercise 1 solution ..... 3
Exercise 2 – Which measures to use? ..... 4
Exercise 3 ..... 4
Exercise 3 solution ..... 5
Exercise 4 – What do these summary statistics tell us? ..... 5
Module 2 – Visualisations and skew ..... 6
Exercise 1 – Create a histogram and a scatter plot ..... 6
Exercise 2 – Compare two groups using histograms ..... 6
Exercise 2 solution ..... 7
Exercise 3 – Anomaly spotting on scatter plots ..... 7
Module 3 – Introduction to probability ..... 8
Exercise 1 – Simple probability ..... 8
Exercise 1 solution ..... 8
Exercise 2 – Binomial results: Six or not six ..... 9
Module 4 – Introduction to distributions ..... 10
Exercise 1 – Binomial distribution ..... 10
Exercise 1 solutions ..... 11
Module 5 – Sampling ..... 13
Exercise 1 – Sampling techniques ..... 13
Exercise 2 – Random numbers (Python) ..... 14
Exercise 2 solution ..... 14
Exercise 3 – Sampling (Python) ..... 15
Exercise 3 solutions ..... 16
Module 6 – Introduction to Hypothesis testing ..... 17
Exercise 1 – Conducting a T-test ..... 17
Exercise 1 solution ..... 17
Exercise 2 – Significance level ethics ..... 18
Module 7 – Linear regression ..... 19
Exercise 1 – Potential use cases ..... 19
Exercise 2 – Interpreting code (Python) ..... 20

# Module 1 – Central tendency, variation, and outliers

### Exercise 1 – Python

- Import pandas as pd
- Create a list of numbers: data = [...] and use pd.Series(data) to save it as df.
- Use the method .mean(), .var(), and .std() to calculate the mean, variance, and standard deviation of your data
- Calculate the outlier boundaries using mean – 2 * sd and mean + 2 * sd
- Check if there are any outliers in your data (values that are smaller than the lower bound or larger than the upper bound)

### Exercise 1 solution

```python
import pandas as pd

data = [2, 3, 4, 6, 7, 7, 8, 19]
df = pd.Series(data)

mean = df.mean()
print('Mean:', mean)

variance = df.var(ddof=0)  # ddof=0 for population variance
standard_deviation = df.std(ddof=0)    # ddof=0 for population standard deviation
print("Population Variance:", variance)
print("Population Standard Deviation:", standard_deviation)

lower_bound_method1 = mean - 2 * standard_deviation
upper_bound_method1 = mean + 2 * standard_deviation
outliers_method1 = [x for x in data if x < lower_bound_method1 or x > upper_bound_method1]
print('Outliers:',outliers_method1)
```

```
Mean: 7.0
Population Variance: 24.5
Population Standard Deviation: 4.949747468305833
Outliers: [19]
```

## Exercise 2 – Which measures to use?

Which measure would best answer the question 'What's a typical value' in each of these scenarios?

1. Test scores in a class
2. Eye colour in a survey of a population
3. Income of employees in a company
4. Age of participants in a marathon
5. Transportation used by people in a city
6. Height of individuals in a study
7. Products purchased in a store

## Exercise 3

1. Use pd.read_csv() to save the investment_performance csv file as a pandas dataframe named df
2. View the data. The scores are results from two investment strategies – the control group (using the current method) and the experimental group (using a new strategy)
3. Using df.describe() or otherwise, compare one measure of the 'typical' value for the two groups and one measure of how consistent or spread out the data is within each group

## Exercise 3 solution

```
import pandas as pd
import numpy as np

# Load data from the CSV file using pandas
csv_filename = 'investment_performance.csv'
df = pd.read_csv(csv_filename)

# Calculate statistics for 'Control Scores'
control_mean = df['Control Scores'].mean()
control_mode = df['Control Scores'].mode().iloc[0]
control_median = df['Control Scores'].median()
control_mid_range = (df['Control Scores'].min() + df['Control Scores'].max()) / 2
control_range = df['Control Scores'].max() - df['Control Scores'].min()
control_iqr = np.percentile(df['Control Scores'], 75) - np.percentile(df['Control Scores'], 25)
control_population_variance = df['Control Scores'].var(ddof=0)
control_population_stddev = df['Control Scores'].std(ddof=0)

# Calculate statistics for 'Experimental Scores'
experimental_mean = df['Experimental Scores'].mean()
experimental_mode = df['Experimental Scores'].mode().iloc[0]
experimental_median = df['Experimental Scores'].median()
experimental_mid_range = (df['Experimental Scores'].min() + df['Experimental Scores'].max()) / 2
experimental_range = df['Experimental Scores'].max() - df['Experimental Scores'].min()
experimental_iqr = np.percentile(df['Experimental Scores'], 75) - np.percentile(df['Experimental Scores'], 25)
experimental_population_variance = df['Experimental Scores'].var(ddof=0)
experimental_population_stddev = df['Experimental Scores'].std(ddof=0)
```

## Exercise 4 – What do these summary statistics tell us?

```
Control Scores:
Mean: 78.08751663789478
Mode: 61.135907012612826
Median: 77.38615374332923
Mid-Range: 78.24753427263798
Range: 34.2232545200503
IQR: 15.577896589435682
Population Variance: 89.78049627551594
Population Standard Deviation: 9.47525705590703

Experimental Scores:
Mean: 85.31905514596525
Mode: 66.15394543335037
Median: 84.37171508463862
Mid-Range: 88.38986419909233
Range: 44.47183753148393
IQR: 17.49185513359525
Population Variance: 119.82727219584062
Population Standard Deviation: 10.946564401484176
```

# Module 2 – Visualisations and skew

### Exercise 1 – Create a histogram and a scatter plot

Use the below code to import packages (remove any you don't intend to use) and to generate some random data for use in subsequent questions.

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

# Create a sample dataset
np.random.seed(42)
n_points = 100
x = np.random.rand(n_points)
y = 2 * x + np.random.normal(0, 0.2, n_points)

# Creating a Pandas DataFrame
df = pd.DataFrame({'X': x, 'Y': y})
```

- Using this dataframe, df, create a histogram for column x.
- Using the same dataframe, create a scatter plot.

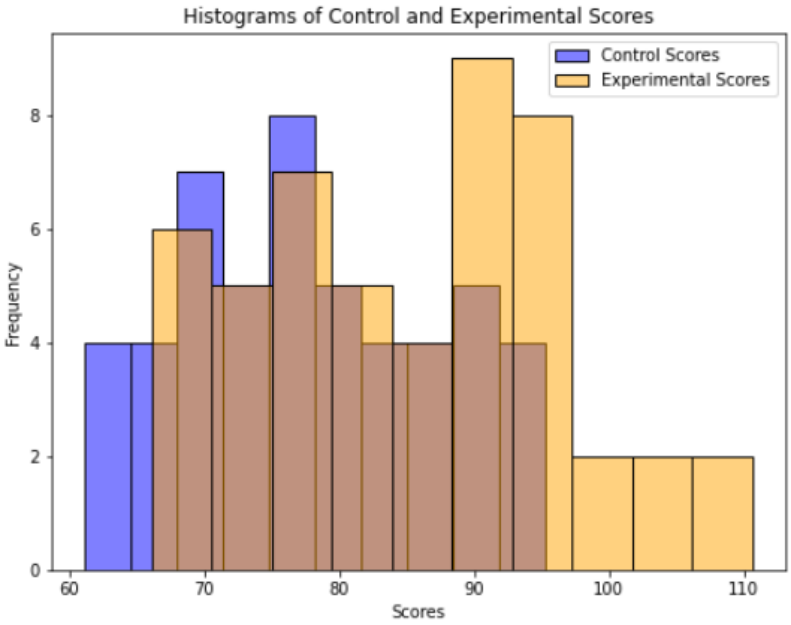
### Exercise 2 – Compare two groups using histograms

1. Import pandas as pd and use pd.read_csv() to save the investment_performance csv file as a pandas dataframe named df.
2. View the data. The scores are results from two investment strategies – the control group (using the current method) and the experimental group (using a new strategy).
3. Import seaborn as sns and matplotlib.pyplot as plt. Use part, or all of the code below to create histograms for the control and experimental groups. What does the visualisation tell you?

```
plt.figure(figsize=(8, 6))
sns.histplot(data=df, x='Control Scores', bins=10, color='blue', label='Control Scores', alpha=0.5)
sns.histplot(data=df, x='Experimental Scores', bins=10, color='orange', label='Experimental Scores', alpha=0.5)
plt.title('Histograms of Control and Experimental Scores')
plt.xlabel('Scores')
plt.ylabel('Frequency')
plt.legend()
plt.show()
```

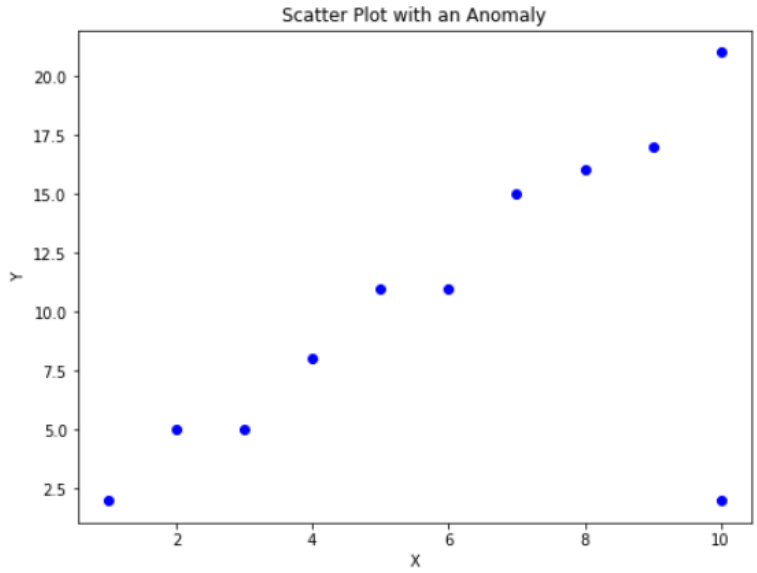
## Exercise 2 solution

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Load data from the CSV file using pandas
csv_filename = 'investment_performance.csv'
df = pd.read_csv(csv_filename)
```




## Exercise 3 – Anomaly spotting on scatter plots

Is there any data that looks out of place?




If the x and y coordinates for the point aren't detected as outliers, how might an algorithm work that could detect anomalies like this one?

# Module 3 – Introduction to probability

## Exercise 1 – Simple probability

- Import random.
- Use rollResult = random.randint(... , ... ) with appropriate values to simulate rolling a 6-sided die. Display rollResult to see what you got.
- To simulate repeated rolling of the die: Initialise an empty list: results = [ ] and then use a for loop to repeat the calculation for rollResult followed by results.append(rollResult). Roll the die as many times as you like by altering how many loops your for loop takes.

```
#For loop syntax, for reference:
for count in range(3):
    print('Ho')
```

```
Ho
Ho
Ho
```

- Use results.count(1) to count how many 1's were rolled.
- Count how many 6's were rolled.
- Calculate the probability of rolling a 1 or rolling a 6 based on the simulation.

## Exercise 1 solution

```

import random

# Simulate rolling a 6-sided die 5 times and record the results
num_rolls = 5
results = []

for count in range(num_rolls):
    roll_result = random.randint(1, 6)
    results.append(roll_result)

# Display the results of the rolls
print(f"Results of: {num_rolls}, Rolls:{results}")

# Calculate the probability of getting a 1 or a 6
count_1 = results.count(1)
count_6 = results.count(6)

probability_1 = count_1 / num_rolls
probability_6 = count_6 / num_rolls

print("Probability of getting a 1:", probability_1)
print("Probability of getting a 6:", probability_6)
print("Probability of getting a 1 or a 6:", probability_1 + probability_6)

```

```
Results of: 5, Rolls:[5, 5, 1, 3, 4]
Probability of getting a 1: 0.2
Probability of getting a 6: 0.0
Probability of getting a 1 or a 6: 0.2
```

### Exercise 2 – Binomial results: Six or not six

- Re-create the code below for yourself.

```
import math

n = 3  # Total number of rolls
r = 1  # Number of S's

combination = math.comb(n, r)
print(f"Number of routes: {combination}")
```

```
Number of routes: 3
```

# Module 4 – Introduction to distributions

### Exercise 1 – Binomial distribution

Look at the answer to the first example and try to answer the remaining examples and define p and X in words, and as a distribution.

- There are 20 parts sampled from a manufacturing run – each either fails quality control or it passes. The probability is the same each time: 0.1 chance of failing.
- In a market research investigation, a focus group of 18 people are each independently asked if they like a product or not. The probability is 50:50 each time.
- A day trader makes 15 trades a day – each is either profitable or not. The probability is the same for each one: 0.9 chance of being profitable.

Extension: Write code to visualise the distribution.

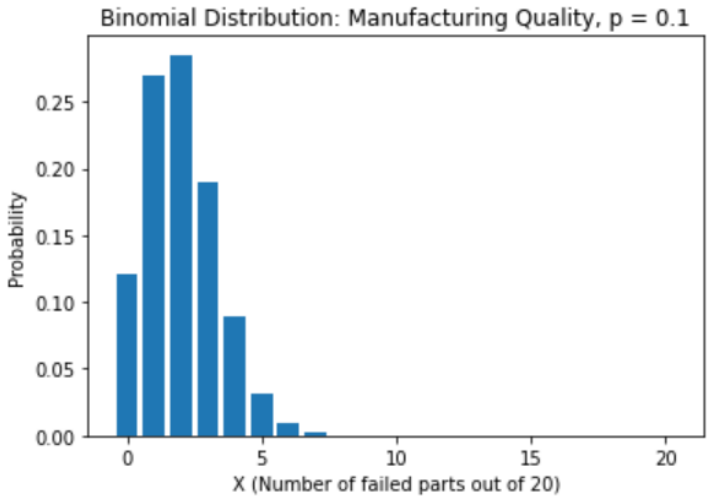
## Exercise 1 solutions

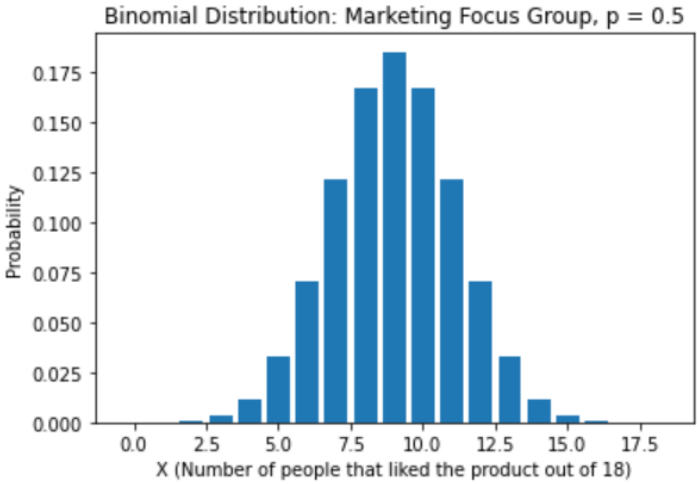
- X ~ B(20, 0.1)

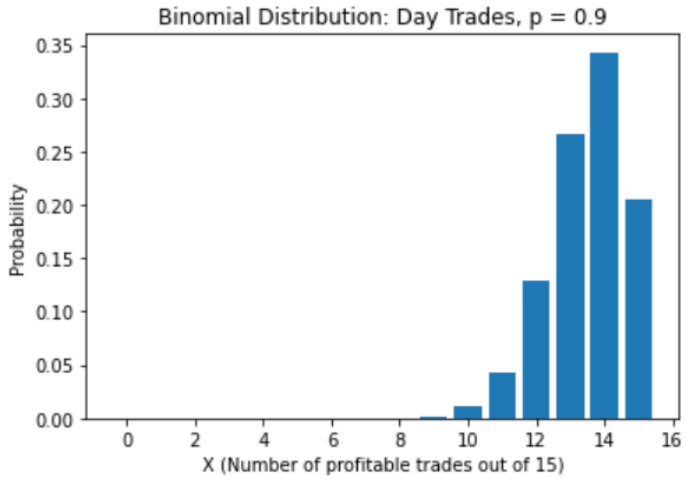
p is the probability that a randomly selected part (out of all the possible parts) fails quality control. X is the number of parts that fail quality control out of 20.

Extension solutions:

Notice the different skews for different values of p. Which looks like a Normal distribution?

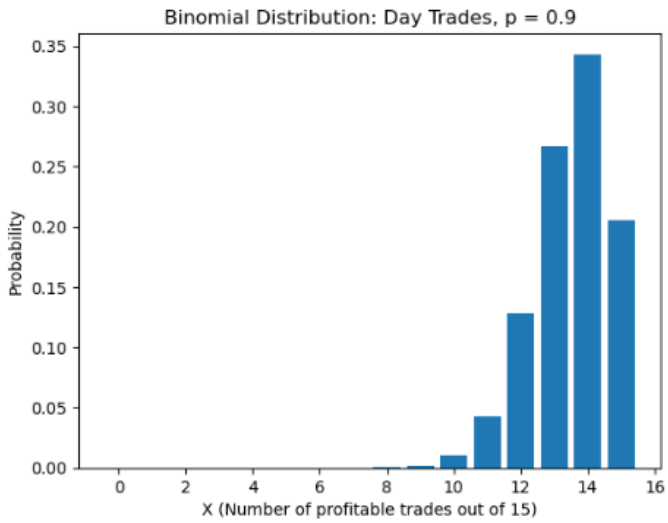





In [24]:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import binom

n = 15  # Number of trades
p = 0.9  # Probability of success each time

# Define the binomial distribution
binomial_dist = binom(n, p)

# Calculate probabilities for different values of X
x_values = np.arange(0, n+1)  # Values of X: 0, 1, 2, ..., n
probabilities = binomial_dist.pmf(x_values)

# Create a bar plot to visualize the probability distribution
plt.bar(x_values, probabilities)
plt.xlabel('X (Number of profitable trades out of 15)')
plt.ylabel('Probability')
plt.title('Binomial Distribution: Day Trades, p = 0.9')
plt.show()
```



# Module 5 – Sampling

### Exercise 1 – Sampling techniques

As a group research and critique each sampling technique:

- Simple Random sampling
- Stratified sampling
- Systematic sampling
- Cluster sampling
- Convenience sampling
- Purposive (Judgmental) sampling
- Snowball sampling

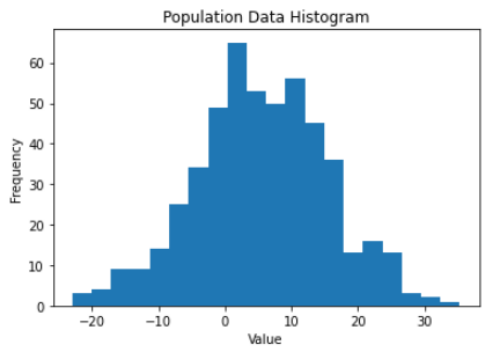
## Exercise 2 – Random numbers (Python)

- Import numpy as np and matplotlib.pyplot as plt
- Use np.random.normal() and choose the values of these arguments: 'loc' is the mean, 'scale' is the standard deviation, and 'size' is the number of values to generate.
- Create 500 normally distributed random numbers with a mean of 5, a standard deviation of 10. Save this as populationData
- Use plt.hist() to visualise the population data.
- Use np.mean() and np.std() to work out the actual mean and standard deviation that was generated.

Note: In real life you may not be able to analyse the whole population – it might mean chopping down every Christmas tree early, opening every bottle of wine in the vineyard, or trying to analyse every social media post from all of your potential customers – not very practical!

## Exercise 2 solution

```
import numpy as np
import matplotlib.pyplot as plt

mean_to_generate = 5
sd_to_generate = 10

np.random.seed(1)
population_data = np.random.normal(loc = mean_to_generate, scale = sd_to_generate, size=500)

plt.hist(population_data, bins=20)  # Adjust the number of bins as needed
plt.title('Population Data Histogram')
plt.xlabel('Value')
plt.ylabel('Frequency')
plt.show()

population_mean = np.mean(population_data)
population_sd = np.std(population_data)

print('Population mean:', population_mean)
print('Population sd:', population_sd)
```




```
Population mean: 5.5343689425630656
Population sd: 9.8840825928265
```

### Exercise 3 – Sampling (Python)

- Use np.random.choice(populationData, size = … ). Save the sample as sample.
- Use np.mean(sample), np.std(sample), and np.std(sample, ddof = 1) to work out the mean and standard deviation of sample. And plt.hist() (followed by plt.show()) to visualise the sample.
- Compare: are the statistics from the sample similar to the results from the population?
- Which standard deviation calculation was closer? (with the default setting or with ddof=1?)

Extension: Repeated sampling and averaged results

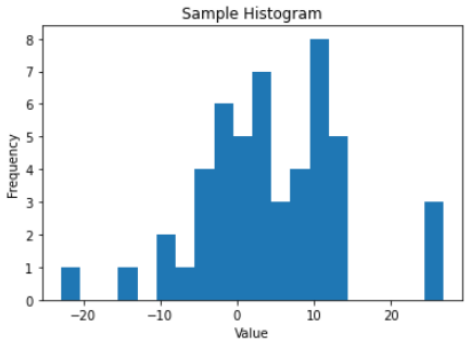
Warning! Do not repeat the display of each sample's histogram!

- Create (initialize) 3 empty lists that will be used to store the sample means, standard deviation, and standard deviation with ddof=1.
- Create a for loop and within it: take a sample (as above) and use the list method append() to save the mean, sd, sdddof1 each time e.g., sampleMeans.append(np.mean(sample)).
- Use np.mean() to work out the average of all the sample means (in the now complete list), the average of the sample sd's, and the average of the sample sdddof1's.
- Is the mean of the samples a good estimate for the actual population? Which standard deviation is closer?

## Exercise 3 solutions

### 1 sample:

```
np.random.seed(1)
sampleSize = 50
sample = np.random.choice(population_data, size = sampleSize)

plt.hist(sample, bins=20)
plt.title('Sample Histogram')
plt.xlabel('Value')
plt.ylabel('Frequency')
plt.show()

sample_mean = np.mean(sample)
sample_sd1 = np.std(sample)
sample_sd2 = np.std(sample, ddof = 1)


print('Sample mean:',sample_mean, '. Compared to the population:', population_mean)
print('Sample sd default:',sample_sd1, '. Compared to the population:', population_sd)
print('Sample sd ddof=1:',sample_sd1, '. Compared to the population:', population_sd)
```




```
Sample mean: 4.357737988094979 . Compared to the population: 5.5343689425630656
Sample sd default: 9.270737308173693 . Compared to the population: 9.8840825928265
Sample sd ddof=1: 9.270737308173693 . Compared to the population: 9.8840825928265
```

### Repeat sample:

```
np.random.seed(1)

number_of_samples = 100000
sample_size = 50

sample_means = []
sample_sd_p = []
sample_sd_s = []

for count in range(number_of_samples):
    sample = np.random.choice(population_data, size = sample_size)
    sample_means.append(np.mean(sample))
    sample_sd_p.append(np.std(sample))
    sample_sd_s.append(np.std(sample, ddof = 1)) #setting ddof = 1 means the n-1 formula will be used.

print("The sample means average out as", np.mean(sample_means), 'compared to the population:', population_mean)
print("The samples standard deviations (n formula) average out as",
      np.mean(sample_sd_p),
      'and for the n-1 formula:',
      np.mean(sample_sd_s), 'compared to the population:', population_sd)
```

```
The sample means average out as 5.5272770592644465 compared to the population: 5.5343689425630656
The samples standard deviations (n formula) average out as 9.73202902243769 and for the n-1 formula: 9.830833880671399 compared
to the population: 9.8840825928265
```

# Module 6 – Introduction to Hypothesis testing

### Exercise 1 – Conducting a T-test

- Import pandas as pd and from scipy import stats.
- Use pd.read_csv() to save the data in investment_performance.csv as df, a pandas dataframe.
- Use stats.ttest_ind() with each of the two columns from the dataframe addressed in the brackets.
- Compare the p-value to half the significance level (since it's a two tailed test).
- Conclude: What to do about the assumption that was made (That the null hypothesis is true)?
- Conclude in context: What does that mean we have evidence for in this particular context?

### Exercise 1 solution

```
import pandas as pd
from scipy import stats

csv_filename = 'investment_performance.csv'
df = pd.read_csv(csv_filename)
stats.ttest_ind(df['Control Scores'], df['Experimental Scores'])
```

```
Ttest_indResult(statistic=-3.496431755693145, pvalue=0.0007101500664096571)
```

Comparison: 0.000710 (3 s.f.) < 0.025

This is very unlikely to have happened if the assumption was correct.

The result is significant. Reject the null hypothesis and therefore accept the alternative.

In context: There seems to be enough evidence to suggest there is a difference between the two investment strategies. Notice that the conclusion does not promise certainties!

### Exercise 2 – Significance level ethics

A test is done for an illness, assuming the person doesn't have it.

Is it better to use a higher significance level?

- More chance of a false positive (rejecting the null hypothesis incorrectly)
- If this happens, further tests are done.

Is it better to use a lower significance level?

- More chance of a false negative (accepting, sticking with, the null hypothesis incorrectly)
- If this happens, they go home, and no further action is taken.

# Module 7 – Linear regression

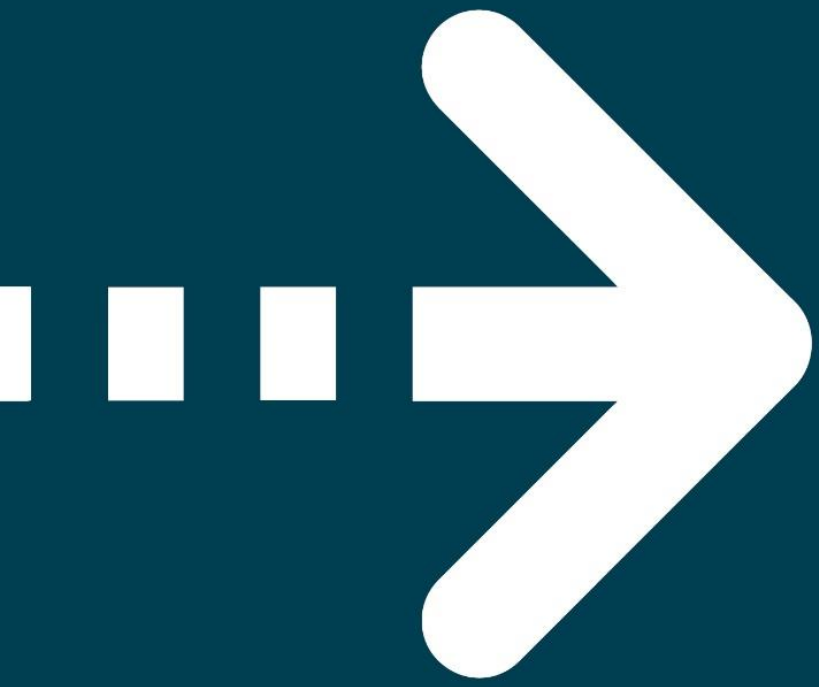
### Exercise 1 – Potential use cases

- What might it be useful to predict?
- What other variables do you suspect might be connected?

Note: If you find yourself thinking of examples where you want to predict 'yes' or 'no' rather than a variety of continuous numbers – you will be interested in classification models rather than linear regression. Make a note of your ideas for later use.

## Exercise 2 – Interpreting code (Python)

What comment would you put to describe what is happening in sections A - I?

```python
# A
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import pearsonr
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score

# B
np.random.seed(0)
x = np.random.rand(50) * 10
y = 2 * x + 1 + np.random.randn(50) * 2

# C
corr_coeff, p_value = pearsonr(x, y)
correlation_text = f"r = {corr_coeff:.2f} and p-value = {p_value}"

# D
x = x.reshape(-1, 1)
y = y.reshape(-1, 1)

# E
model = LinearRegression()
model.fit(x, y)
equation_text = f"y = {model.coef_[0][0]:.2f}x + {model.intercept_[0]:.2f}"

# F
r_squared = r2_score(y, model.predict(x))
r_squared_text = f"R Squared = {r_squared:.2f}"

# G
residuals = y - model.predict(x)
std_dev = np.std(residuals)
two_std = 2 * std_dev
```

```python
# H
plt.figure(figsize=(10, 6))
plt.scatter(x, y, label="Data points", color="blue")
plt.plot(x_pred, y_pred, color="red", label="Linear Regression")
plt.fill_between(
    x_pred.flatten(),
    y_pred.flatten() - two_std,
    y_pred.flatten() + two_std,
    color="gray",
    alpha=0.3,
    label="Error Window",
)
plt.legend()
plt.grid(True)
plt.show()

# I
print(correlation_text)
print(equation_text)
print(r_squared_text)
```